

Week - 7

Q1:

```
q1.cu  X
Week7 > q1.cu > countWord(char *, char *, int *, int, int)
1  #include <stdio.h>
2  #include <string.h>
3
4  global void countWord(char *sentence, char *word, int *count, int n, int m)
5  {
6      int idx = blockIdx.x * blockDim.x + threadIdx.x;
7
8      if (idx <= n - m) {
9          int match = 1;
10
11          for (int j = 0; j < m; j++) {
12              if (sentence[idx + j] != word[j]) {
13                  match = 0;
14                  break;
15              }
16          }
17
18          if (match) {
19              atomicAdd(count, 1);
20          }
21      }
22  }
23
24  int main() {
25      char sentence[] = "cuda is fast and cuda is powerful cuda";
26      char word[] = "cuda";
27
28      int n = strlen(sentence);
29      int m = strlen(word);
30
31      char *d_sentence, *d_word;
32      int *d_count;
33      int h_count = 0;
34
35      cudaMalloc((void**)&d_sentence, n);
36      cudaMalloc((void**)&d_word, m);
37      cudaMalloc((void**)&d_count, sizeof(int));
38
39      cudaMemcpy(d_sentence, sentence, n, cudaMemcpyHostToDevice);
40      cudaMemcpy(d_word, word, m, cudaMemcpyHostToDevice);
41      cudaMemcpy(d_count, &h_count, sizeof(int), cudaMemcpyHostToDevice);
42
43      int threads = 256;
44      int blocks = (n + threads - 1) / threads;
45
46      countWord<<<blocks, threads>>>(d_sentence, d_word, d_count, n, m);
47
48      cudaMemcpy(&h_count, d_count, sizeof(int), cudaMemcpyDeviceToHost);
49
50      printf("Occurrences of '%s': %d\n", word, h_count);
51      printf("Adarsh Ranjan, 230962278");
52
53      cudaFree(d_sentence);
54      cudaFree(d_word);
55      cudaFree(d_count);
56
57      return 0;
58  }
```

```

(base) mca@computinglab25-22:~/Desktop/PPL_230962278/Week7$ nvcc q1.cu -o q1
(base) mca@computinglab25-22:~/Desktop/PPL_230962278/Week7$ ./q1
Occurrences of 'cuda': 3
Adarsh Ranjan, 230962278(base) mca@computinglab25-22:~/Desktop/PPL_230962278/Week7$

```

Q2:

```

q2.cu
Week7 > q2.cu > main()
1  #include <stdio.h>
2  #include <string.h>
3
4  global void buildRS(char *S, char *RS, int n) {
5      int i = threadIdx.x;
6
7      if (i < n) {
8          int pos = 0;
9
10         for (int k = 0; k < i; k++) {
11             pos += (k + 1);
12         }
13
14         for (int j = 0; j <= i; j++) {
15             RS[pos + j] = S[j];
16         }
17     }
18 }
19
20 int main() {
21     char S[] = "PCAP";
22     int n = strlen(S);
23
24     int total_len = n * (n + 1) / 2;
25
26     char *d_S, *d_RS;
27     char RS[100];
28
29     cudaMalloc((void**)&d_S, n);
30     cudaMalloc((void**)&d_RS, total_len);
31
32     cudaMemcpy(d_S, S, n, cudaMemcpyHostToDevice);
33
34     buildRS<<<1, n>>>(d_S, d_RS, n);
35
36     cudaMemcpy(RS, d_RS, total_len, cudaMemcpyDeviceToHost);
37     RS[total_len] = '\0';
38
39     printf("Input S: %s\n", S);
40     printf("Output RS: %s\n", RS);
41     printf("Adarsh Ranjan, 230962278");
42     cudaFree(d_S);
43     cudaFree(d_RS);
44
45     return 0;
46 }

```

```
● ^[[A(base) mca@computinglab25-22:~/Desktop/PPL_230962278/Week7$ ./q2
Input S: PCAP
Output RS: PPCPCAPCAP
○ Adarsh Ranjan, 230962278(base) mca@computinglab25-22:~/Desktop/PPL_230962278/Week7$
```